

CSCE 625 ARTIFICIAL INTELLIGENCE · MODULE 05 · LECTURE NOTES

Adversarial Search

Searching when someone else chooses too.

MODULE OUTCOMES

- Explain minimax and its assumptions.
- Explain alpha-beta pruning and why move ordering decides its value.
- Explain evaluation functions and the horizon effect.
- Explain Monte Carlo tree search and when it beats minimax.
- Choose between the two for a given game.

1 Minimax and alpha-beta

MINIMAX ASSUMES A PERFECT OPPONENT, AND THAT ASSUMPTION IS A CHOICE

At your own nodes take the maximum over children; at the opponent's take the minimum. The resulting value is what you can guarantee against best play, which is a meaningful and well-defined thing to compute.

This is the right assumption against a strong opponent and a conservative one against a weak one.

Minimax will never set a trap that only works if the opponent blunders, because it assumes they will not — which makes it play solidly and unimaginatively against weak players.

Which is exactly why game AI frequently does not want minimax. A game opponent should be beatable and interesting, not optimal, and minimax provides no knob for either (§4).

The cost is $O(b^d)$ and the complete tree is almost never searchable — chess has on the order of 10^{40} reachable positions — **so in practice you search to a fixed depth and apply a heuristic evaluation there** (§2), which is where the method's real difficulties live.

```
alpha = the best value MAX can already guarantee
beta  = the best value MIN can already guarantee
```

```
While searching a MIN node, if its value drops to or below
alpha, MAX will never choose this branch -- so STOP. The
remaining children cannot change the decision above.
```

```
Symmetrically for MAX nodes against beta.
```

```
THE RESULT IS EXACT. Alpha-beta returns the SAME value as
full minimax; it only skips nodes that provably cannot
affect it. This is not an approximation.
```

```
THE SPEEDUP DEPENDS ENTIRELY ON MOVE ORDERING:
```

```
worst ordering  ->  $O(b^d)$       no saving whatsoever
random ordering ->  $O(b^{(3d/4)})$ 
BEST ordering   ->  $O(b^{(d/2)})$   TWICE THE DEPTH for the
                                same node count
```

```
SO THE ENGINEERING IS IN THE ORDERING, not the pruning:
```

```
try the previous iteration's best move first --
    iterative deepening (Module 02) hands you this
    for free, which is why engines use it
killer moves: moves that caused a cutoff elsewhere at
    the same depth
captures and threats before quiet moves
a transposition table, which also catches positions
    reached by different move orders -- the explored
    set of Module 02, in a game tree
```

```
A chess engine's strength is mostly move ordering and
evaluation. The search algorithm itself is thirty lines.
```

Doubling the searchable depth for the same node count is the single largest result in game-tree search, and it is entirely contingent on ordering — which is why so much engine engineering goes into move ordering heuristics that look, from outside, like minor details. **The lesson generalises: the algorithm with the famous name is often the easy part.**

2 Evaluation and the horizon

THE EVALUATION FUNCTION AND THE HORIZON EFFECT

Search to depth d , then score the resulting position with a heuristic evaluation — material balance, piece placement, mobility, pawn structure, king safety. The quality of this function is most of the engine's strength.

The horizon effect: a bad event just beyond the search depth is invisible to the search, so the agent makes pointless delaying moves that push the event one ply further out, at which point it vanishes from view entirely. **The agent has not avoided the loss; it has avoided seeing it**, and it has spent material doing so.

Quiescence search is the fix: extend the search beyond the nominal depth limit while the position remains volatile — captures available, checks outstanding, promotions pending — **and evaluate only quiet positions**.

Because evaluating a position mid-exchange is meaningless: you appear to be a queen ahead because the recapture happens to lie one ply past your limit. **Quiescence search is not optional in any real engine**, and an engine without it plays recognisably badly in a specific way — which is a good illustration that the headline algorithm's correctness does not imply the system's.

3 Monte Carlo tree search

```

repeat until the time budget is exhausted:
  SELECT  descend the existing tree, at each node
          choosing the child maximising
           $UCB1 = \text{mean\_value} + c \cdot \sqrt{\ln(N)/n}$ 
          which balances EXPLOITING children with good
          observed value against EXPLORING ones with
          few samples
  EXPAND  add one new child beyond the tree
  SIMULATE play to the end of the game, randomly or
          with a cheap policy
  BACKUP  propagate the outcome up the visited path

```

then play the most-visited child.

WHY THIS DIFFERS FROM MINIMAX

NO EVALUATION FUNCTION IS NEEDED -- the simulation's outcome IS the evaluation. Decisive for games where nobody knows how to evaluate a position, which was the situation in Go for forty years.

ANYTIME -- stop at any moment; the answer degrades smoothly rather than being unavailable. Exactly the right shape for a frame budget (Module 04 section 3).

ASYMMETRIC -- it deepens promising lines and ignores bad ones, rather than searching to uniform depth.

TOLERATES HIGH BRANCHING -- it samples rather than enumerating, so $b = 250$ is survivable where alpha-beta is not.

AND IT IS WEAKER where a good evaluation exists and tactics are sharp: random playouts miss forced sequences that alpha-beta finds exactly. Top chess engines remain alpha-beta based for this reason.

UCB1 is the explore-exploit trade from the multi-armed bandit literature, and it reappears in CSCE 642 §2 as the same formula for the same purpose — which is worth noticing, because it means the exploration problem in reinforcement learning and the selection problem in game-tree search are literally the same problem with different payoffs.

Choose minimax with alpha-beta when	Choose MCTS when
A good evaluation function exists and is cheap.	Nobody knows how to evaluate a position reliably.
The branching factor is modest — chess is about 35.	The branching factor is large — Go is about 250.
Tactics are sharp and forced sequences decide games.	The position is strategic and advantages accumulate gradually.
You can search to a useful fixed depth within budget.	You need an anytime answer under an external deadline.
The game is deterministic and fully observable.	The game is stochastic — simulate the randomness directly, which minimax handles only awkwardly.

And modern strong play is both. AlphaZero is MCTS with a learned evaluation function and a learned move prior — which removes MCTS's central weakness by supplying exactly what it lacked: the prior guides selection away from nonsense moves, and the learned evaluation replaces the random payout. **The synthesis is the right place to leave this**, and it is also a good example of a learned component slotting into a classical algorithm rather than replacing it (CSCE 753 Module 03 §4's pattern).

4 Adversarial search in a game

AN OPTIMAL OPPONENT IS USUALLY THE WRONG PRODUCT

Players want an opponent that is challenging, readable, and beatable — not one that is optimal. **An unbeatable opponent is a solved design problem and a failed product.**

So difficulty is tuned by handicapping the search deliberately: a shallower depth limit, a noisier evaluation function, an occasional deliberate suboptimal choice, or a restricted set of available tactics at lower difficulties.

Add the noise to the evaluation rather than to the move choice. A noisy evaluation function makes plausible mistakes — it misjudges a position the way a weaker player would — **while a randomly perturbed move choice makes absurd ones**, and **players notice the difference immediately and describe the second as broken rather than as easy**. This is one of the most useful pieces of practical knowledge in game AI.

And make the agent's reasoning legible. An opponent whose plan the player can infer and counter is more satisfying than a stronger opaque one — which means **legibility is a design constraint on the algorithm rather than an afterthought**, and it is another reason game AI favours the methods of this course (Module 01 §4's debuggability argument, from the player's side rather than the developer's).

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Russell & Norvig — AIMA, chapter 5 (pseudocode free)	Minimax, alpha-beta, and the ordering results of §1 , with the complexity derivations.
Chess Programming Wiki (free)	The practical reference for §1 and §2 — move ordering, transposition tables, and quiescence search, written by engine authors.
Browne et al. — A Survey of Monte Carlo Tree Search Methods (free)	The §3 reference , with UCB1 derived and the variants catalogued.
Silver et al. — Mastering the Game of Go without Human Knowledge (free)	The synthesis of §3's table footnote — MCTS with learned evaluation and prior.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement minimax** for a small game and verify the value against exhaustive search.
- 2 **Add alpha-beta** and confirm the value is unchanged while the node count drops.
- 3 **Measure nodes expanded under worst, random, and best move ordering** and compare against the predicted exponents.
- 4 **Add iterative deepening with a best-move-first ordering** and report the improvement.
- 5 **Add a transposition table** and report the hit rate.
- 6 **Construct a horizon-effect position** and show your engine making a delaying move.
- 7 **Add quiescence search** and confirm the behaviour changes.
- 8 **Implement MCTS** for the same game and compare against alpha-beta at equal time budgets.
- 9 **Vary the UCB1 exploration constant** and plot playing strength against it.
- 10 **Build three difficulty levels** — by depth, by evaluation noise, and by random move choice — and have someone play all three without being told which is which. Report what they said.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does minimax assume, and why is that sometimes wrong for a game?
- 2 Explain alpha-beta and state that its result is exact.
- 3 Give the three ordering complexities and what follows from them.
- 4 Name four move-ordering techniques.
- 5 What is the horizon effect, and what fixes it?
- 6 Why is evaluating a mid-exchange position meaningless?
- 7 Give MCTS's four steps and what UCB1 balances.
- 8 Give four MCTS advantages and its main weakness.
- 9 Compare minimax and MCTS on five axes.
- 10 Why add noise to the evaluation rather than the move choice?